Experiment 6: File Manipulation Using Shell Scripts

Copy, Comparison and Pattern-Matching Operations

Based on the practical list in your image, Experiment 6 is:

Aim:“To create and manipulate files using shell scripts involving copy, comparison, and pattern-matching operations.”

1. File creation
2. File copying
3. File comparison
4. Pattern matching
5. Searching text using grep
6. Displaying and manipulating file content

2. Objectives

After completing this experiment, students will be able to:

* Create files using Linux commands.
* Write data into files.
* Copy one file into another.
* Compare two files.
* Check whether two files are identical.
* Search a particular pattern in a file.
* Use wildcards such as *, ?, and [].
* Use grep for pattern matching.
* Write a shell script for file operations.
* Understand exit status of Linux commands.

3. Theory

A file in Linux is used to store information such as text, program code, configuration data, etc.Shell scripting allows us to automate file-related operations.

For example:

Bash

Cmp file1.txt file2.txt

Compares two files.

And:

Bash

grep “Linux” file1.txt

Copying File – cp

The cp command is used to copy files.

Syntax

Bash

Cp source destination

Example

Bash

Cp file.1.txt file2.txt

Here:

File1.txt= Source file

File2.txt=destination file

Now both file contain the same data.

Copy a file to another directory

Bash

Cp file1.txt /home/user/Documents/

Comparing Files

Linux provides several commands for comparison.

Bash

Cmp file1.txt file2.txt

Viva Questions

1. What is the purpose of the cp command?

2. What is the syntax of the cp command?

3. Which command is used to compare two files?

4. What is the difference between cmp and diff?

5. What is pattern matching?

6. Which command is used for pattern matching?

7. What is the purpose of the grep command?

Experiment -7

Aim: To write and execute a Shell Script to display various system-related information, including:

* Logged-in users
* Current user
* Current date and time
* File and directory listings
* Running jobs
* Background processes
* Process information
* System information

—

2. Objectives

After completing this experiment, students will be able to:

1. Display the currently logged-in users.
2. Identify the current user.
3. Display date and time.
4. List files and directories.
5. Understand foreground and background processes.
6. Display jobs running in the current shell.
7. Display running processes.

8. Understand process IDs.

9. Use shell commands inside a script.

10. Automate system-information collection using Bash.

3. Theory

A process is a program that is currently running in the operating system.

For example, when we open a terminal, a shell process is running. When we execute a command, that command also becomes a process.

Linux provides several commands for monitoring users, files, jobs, and processes.

Important commands for this experiment are:

who

The who command displays information about users currently logged into the system.

Bash

Who

Example

Plain text

Student pts/0 2026-08-31

Student pts/1 2026-08-31

It can show:

- Username
- Terminal
- Login date
- Login time

whoami

The whoami command displays the username of the current user

Bash

whomi

Example:

Student

Command	**Purpose**
Who	Shows logged-in users
Whoami	Shows the current user

date

The date command displays the current system date and time

Bash

date

plain text

Mon Aug 31 10:15:30 IST 2026

Viva Questions:

1.Which command shows currently logged-in users along with their active processes and system uptime?

2. What is the difference between the who, w, and whoami commands in Linux?

3. How do you send a process to run in the background, and which command lists all current jobs running in the background of a terminal session?

4. What is the difference between ps, top, and htop for monitoring running processes?

5. How do you bring a background process back to the foreground or force-terminate it using its Process ID (PID)?

6. Which options used with the ls command allow you to display file permissions, ownership, size, and hidden files?

7. Which environment variables and system commands can be used in a shell script to display basic system information like system kernel version, memory usage, and the current user?

Expeiment 8

Aim

To write a shell script to change/format the system date display and measure the execution time of a script.

Theory & Commands Used

date command: Displays or sets the system date and time. It allows custom formatting using format specifiers (e.g., %Y for year, %m for month, %d for day, %H for hour, %M for minute, %S for second).

time command: Measures how long a given command or script takes to execute. It outputs real time (elapsed time), user CPU time, and system CPU time.

SECONDS variable: A built-in shell variable in Bash that counts the number of seconds since the shell script started executing.

Algorithm / Procedure

1. Part A: System Date Formatting

Prompt the user or automatically fetch system date/time.

Format and display the current date into specific custom formats (e.g., YYYY-MM-DD, DD/MM/YYYY, HH:MM:SS).

2. Part B: Script Execution Time Measurement

Record the start time (using date +%s or resetting SECONDS=0).

Execute the target task or workload (e.g., a loop or sleep command).

Record the end time and calculate the difference to find total execution time.

Shell Scripts

Display Date in custom Formats

```
#!/bin/bash


echo "=== System Date & Time Formats ==="

echo "Default Date & Time   : $(date)"

echo "Format (YYYY-MM-DD)   : $(date +%Y-%m-%d)"
```

```
echo "Format (DD/MM/YYYY)   : $(date +%d/%m/%Y)"

echo "Format (Day, Month D) : $(date +'%A, %B %d, %Y')"

echo "Time (12-Hour Format) : $(date +'%I:%M:%S %p')"

echo "Time (24-Hour Format) : $(date +'%H:%M:%S')"
```

Viva Questions :

1. Which format specifiers with the date command display the year in 4-digit format, month number, and day of the month?

2. What is the exact purpose of date +%s in shell scripting, and how is it used to compute elapsed execution time?

3. How does the built-in Bash variable $SECONDS work, and how can it be reset inside a script to track execution time?

4. What is the difference between real, user, and sys time reported by the time command output?

5. What command-line tools and root privileges are required to change or synchronize the actual system date and time in Linux?

6. How can you display the system time in 12-hour format with AM/PM versus 24-hour format using date format strings?

7. What is the difference between measuring script execution time using internal timestamps versus using the external time utility?

Experiment 9

Aim

To write a shell script to display file names in a specified directory along with their creation/modification date and serial (inode) number.

Theory & Commands Used

Inode (Index Node) Number: A unique numerical identifier assigned to each file and directory in a UNIX/Linux file system, storing metadata about the file except for its name and actual content.

ls command:

-i: Displays the inode number (serial number) of each file.

-l: Displays detailed file attributes, including modification/creation date and timestamp.

stat command: Displays detailed status and metadata of a file, including inode number, access time, modification time, and change time.

Algorithm / Procedure

1. Define or accept the target directory path from the user (defaulting to the current directory if omitted).

2. Check if the specified directory exists using shell conditional flags ([-d "$dir"]).

3. Iterate through or list the files using formatted output (ls -li or stat).

4. Display the inode number, date/time stamp, and file name in a clean tabular format.

Shell Script

```
#!/bin/bash


# Prompt user for directory path, or default to current directory

read -p "Enter directory path (leave blank for current directory): " dir


# Use current directory if input is empty

dir=${dir:-.}


# Check if directory exists
```

```
if [ ! -d "$dir" ]; then
    echo "Error: Directory '$dir' does not exist."
    exit 1
fi

echo
"=============================================================================
======"
echo "INODE NUMBER    | DATE & TIME              | FILE NAME"
echo
"=============================================================================
======"

# Loop through all entries in the target directory
for file in "$dir"/*; do
    # Ensure entry exists (handles empty directory case)
    if [ -e "$file" ]; then
        # Extract inode number using stat
        inode=$(stat -c "%i" "$file")

        # Extract last modification date/time
        mod_date=$(stat -c "%y" "$file" | cut -d'.' -f1)

        # Extract filename only
        filename=$(basename "$file")

        printf "%-15s | %-24s | %s\n" "$inode" "$mod_date" "$filename"
    fi
```

```
done


echo
"=================================================================
======"
```

Viva Questions

1. What is an inode (index node) in a Linux file system, and what information does it store?

2. Which flag is used with the ls command to display the inode number of a file?

3. What is the difference between file access time (atime), modification time (mtime), and change time (ctime) in Linux?

4. How does the stat command differ from the standard ls -l command?

5. Does an inode store the filename of a file? (Explain why or why not).

6. How does a hard link relate to a file's inode number compared to a symbolic (soft) link?

7. What format specifiers are used with stat -c to extract inode numbers and modification timestamps?